

Birla Institute of Technology and Science, Pilani
2nd Semester 2024-2025
CS F111 – Computer Programming – Comprehensive Exam – Part A (Closed Book)

5 May 2025

Duration: 60 minutes

33 Marks

[Instructions: There are 22 questions. Q1 to Q11 are for 1M each. Q12 to Q22 are for 2M each. More than one option can be correct, and marks will be awarded only when all the correct options are marked. No Partial or negative marking for any question. Write answers in the separate answer sheet provided. Use the main answer booklet for rough work. Strictly follow the instructions written in the answer sheet while attempting. **End of Instructions]**

1. What does the following statement do? `int (*ptr)[10];`
a) **Creates a pointer to an integer array of size 10** b) Creates an array of 10 integer pointers
c) Creates a pointer that holds 10 addresses d) None of the above
2. What happens when you try to dereference a NULL pointer?
a) **Undefined behavior or Segmentation Fault** b) The program prints NULL
c) The pointer points to zero d) Compilation error
3. Which of the following is a correct way to convert a command-line argument into an integer?
a) `int x = int(argv[1]);` b) `int x = stoi(argv[1]);`
c) **`int x = atoi(argv[1]);`** d) `int x = scanf("%d", argv[1]);`
4. Which of the following statements are TRUE in C?
a) Recursive macros are supported in C. **b) Macros in C are processed by the preprocessor.**
c) **Functions in C can call themselves recursively.** d) Macros in C support type checking at compile time.
5. Which of the following lines of code correctly declares and initialises a file pointer for writing to a file in C?
a) `FILE fp = fopen("output.txt", "w");` b) `file *fp = fopen("output.txt", "w");`
c) **`FILE *fp = fopen("output.txt", "w");`** d) `FILE *fp; fopen("output.txt", "w");`
6. Which of the following improve the performance of a C program?
a) **Using compiler optimisation flags like `-o2`, `-o3` and `-ofast`**
b) Making our code modular by using a larger number of functions
c) Ensure that memory leaks don't occur d) Using Functions in place of Macros
7. Which of the following statements about **recursion** in C is/are **TRUE**?
a) **Every recursive function must have a base condition to prevent infinite recursion.**
b) Recursive functions always execute faster than iterative solutions.
c) **A recursive function calls itself directly or indirectly.** d) The return type of a recursive function must be void.
8. Which of the following correctly describes the behavior of the `calloc()` function in C?
a) It allocates memory without initializing it **b) It allocates memory and initializes all bytes to zero**
c) It reallocates memory to a larger size d) It frees previously allocated memory
9. Which of the following statements about static arrays in C is true?
a) The size of a static array can be changed after its declaration b) Array elements in C are indexed starting from 1
c) **Arrays are stored in contiguous memory locations** d) A static 2-D array can have values from different data types
10. What is the correct signature of the `main()` function to accept command-line arguments in C?
a) `int main()` b) `int main(char *argv[], int argc)`
c) **`int main(int argc, char *argv[])`** d) `void main(int argc, char argv[])`
11. Which of the following statements about circular singly linked lists in C is TRUE?
a) The last node in a circular singly linked list contains a NULL pointer.
b) A circular singly linked list can be traversed using a `while(ptr != NULL)` loop.
c) A circular singly linked list must always contain at least two nodes.
d) **In a circular singly linked list, the last node's next pointer points back to the head node.**

12. What will be the output of the following code?

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, *p, *q;
    p = &a;    q = &b;    *p = *q;
    printf("%d %d", a, b);
    return 0;
}
```

- a) 20 20 b) 10 10 c) 10 20 d) 21 21

13. What will be the output of the following code snippet?

```
#include <stdio.h>
void fun(int *p) {
    *p = *p + 10;
}
int main() {
    int x = 5;    fun(&x);    printf("%d", x);
    return 0;
}
```

- a) 5 b) 10 c) 15 d) Compilation error

14. What will be the output of the following code?

```
int main() {
    char *ptr = "Hello";    *(ptr + 1) = 'a';    printf("%s\n", ptr);
    return 0;
}
```

- a) Hallo b) Compilation error c) Segmentation fault d) Undefined behavior

Either of them can be true.

15. What will be the output of the following program?

```
#include <stdio.h>
int main() {
    int x = 5;
    int *p1 = &x;
    int **p2 = &p1;

    **p2 = 10;
    printf("%d\n", x);
    return 0;
}
```

- a) 5 b) 10 c) Compilation error d) Garbage value

16. What will be the output of the following program?

```
#include <stdio.h>
void update(int **p) {
    static int val = 20;
    *p = &val;
}
int main() {
    int x = 10;    int *ptr = &x;
    update(&ptr);
    printf("%d\n", *ptr);
    return 0;
}
```

- a) 20 b) 10 c) Garbage value d) Compilation error

17. Which of the following code snippets can cause a **memory leak** in C?

a) <code>int *ptr = (int *)malloc(sizeof(int)); *ptr = 10; free(ptr);</code>	b) <code>int *ptr = (int *)malloc(sizeof(int)); ptr = NULL;</code>
c) <code>int arr[10];</code>	d) <code>int *ptr = (int *)malloc(sizeof(int)); *ptr = 5; ptr = (int *)malloc(sizeof(int));</code>

18. Which of the following code snippets can result in a dangling pointer issue?

a) <code>int *ptr = (int *)malloc(sizeof(int)); free(ptr); *ptr = 10;</code>	b) <code>int* getPointer() { int x = 10; return &x; }</code>
c) <code>int x = 5; int *ptr = &x;</code>	d) None of the above

19. What will be the output of the following program?

```
#include <stdio.h>

#define ADD(x) x + x + x
#define FUN(x) ADD(x) * 2
#define NUF(x) x * 2

int main() {
    int a = 5;
    int result1 = FUN(ADD(a));
    int result2 = FUN(NUF(a));
    int result3 = FUN(NUF(ADD(a)));

    printf("R1=%d\n", result1);
    printf("R2=%d\n", result2);
    printf("R3=%d\n", result3);

    return 0;
}
```

a) R1 = 90 R2 = 80 R3 = 130	b) R1 = 50 R2 = 40 R3 = 70
c) R1 = 60 R2 = 50 R3 = 100	d) Compilation error due to macro expansion ambiguity

20. What will be the output of the following program?

```
#include <stdio.h>
#define fact(n) (n==0 ? 1 : n * fact(n-1))

int main() {
    int n = 5;
    int result = fact(n); // Calling the macro to compute factorial
    printf("Factorial of %d is %d\n", n, result);
    return 0;
}
```

- a) Factorial of 5 is 120 b) Factorial of 5 is 0 c) Compile-time error d) Runtime error
- Either of them can be true.**

21. What is the output of the following program?

```
#include <stdio.h>
#define fun(n) (n==0 ? 1 : (n * 2))
#define nuf(n) (n==0 ? 1 : (n - 2))

int add()
{
    int n = 10;
    n = n * 2;
    return n;
}

int main() {
    int n = 5;
    int result = fun(n * nuf(n) * fun(n));
    printf("Result=%d", result);
    return 0;
}
```

- a) Result = 300 b) Result = 100 c) Result = 200 d) Result = 150

22. What is the output of the following program?

```
#include <stdio.h>
int main() {
    FILE *fp;

    fp = fopen("output.txt", "w");
    fputs("Hello\n", fp);
    rewind(fp);
    fputs("Bye", fp);
    fclose(fp);

    fp = fopen("output.txt", "a");
    fputs("Hello", fp);
    rewind(fp);
    fputs("Bye", fp);
    fclose(fp);
    return 0;
}
```

a) Hello ByeHello	b) ByeHello
c) Byelo HelloBye	d) HelloByeBye

05/05/2025

Max. Marks: 33M

Duration: 60 mins

ID No:

Name:

Signature:

Instructions:

- Write the answers only in the space provided. Don't let your answers overflow.
- **The marking is binary. No marks will be given for partially correct answers.**
- Overwritten answers will not be accepted for rechecks.
- Wherever there is a semicolon against a blank, you are supposed to write only one C statement. No marks will be awarded for multiple statements written for such questions.
- Use the supplementary sheet for rough work. Don't overwrite your answers as much as possible.
- No Calculators are allowed for the test. Wrist Watches (analogue or digital) are also not allowed.

Invigilator's**Signature:****Recheck request (Write question number only)**

--	--	--	--	--	--	--	--	--	--

1. Given an incomplete program to copy an array (source) to another (destination). Complete it. [1+1+2+1+1=6M]

```
#include <stdio.h>
void copyArray(int **source, int **destination, int size) {

    *destination = (int *)malloc(size * sizeof(int));
    // allocate memory to the destination array

    if (____*destination == NULL____) {
        printf("Memory allocation failed!\n");
        return;
    }

    for (int i = 0; i < size; i++) {

        (*destination)[i] = (*source)[i];
    }

    // allocate memory to the destination array

}

int main() {
    int size = 5;
    int *source = (int *)malloc(size * sizeof(int));

    if (source == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    int values[] = {1, 2, 3, 4, 5}; // Given array
    for (int i = 0; i < size; i++) {
        source[i] = values[i];
    }
    int *destination = NULL;
    copyArray(&source, &destination, size);
}
```

```

printf("Copied Array:\n");
for (int i = 0; i < size; i++) {
    printf("%d ", destination[i]);
}

free(source);    free(destination);
//Release the memory allocated to both the arrays

```

2. What is the output of the following? [5M]

```

#include <stdio.h>
static int i = 100;

void f2() {
    static int i = 0;
    i = i+10;
    printf("i=%d\n", i);
}

void main() {
    int j;
    for(j = 0; j < 1; j++)
        f2();
        i++;
    for(j = 0; j < 2; j++)
        f2();
        printf("i=%d\n", i);
    for(; j < 2; j++)
        printf("i=%d\n", i);
}

```

Answer:

```

i=10
i=20
i=30
i=101

```

3. Write a **recursive** function `int recLen(char *str)` to determine the length of a string – `str`. **5M**

Answer:

```

int recLen(char *str) {
    // Base case: if we reach the null terminator, length is 0
    if (*str == '\0') {
        return 0;
    }
    // Recursive case: count current character and recurse
    return 1 + recLen(str + 1);
}

```

4. The following performs **String Searching** from an array of strings using linear search. It finds the position of a user-given city name from an array of strings. The program does **NOT** include <string.h> library. Implement the function: `int stringCompare(char*, char*)`; which accepts two strings and returns **0** if both strings are the same, and returns **1** if they are different. **5M**

```
#include <stdio.h>
#define N 6

int stringCompare(char* , char* ) ;

int main()
{
    char *str1[N] = {"Chennai", "Pilani", "Goa", "Kolkata", "Mumbai", "Delhi"};
    char str2[20];
    int temp, pos=-1, h, i;
    printf("\nEnter the second string : ");
    scanf("%s",str2);
    for (int h=0; h<N; h++)
    {
        temp=stringCompare(str1[h], str2);
        if(temp==0)
        {
            pos=h;
            break;
        }
    }

    if(pos>-1)
        printf("The city is found at %d position\n", pos+1);
    else
        printf("The city is not found!\n");

    return 0;
}

int stringCompare(char *a,char *b)
{
    int flag=0;
    while (*a != '\0' && *b != '\0') {
        if (*a != *b)
            return 1; // Characters differ, not equal
        a++;
        b++;
    }

    // If both strings end together, they're equal
    if (*a == '\0' && *b == '\0')
        flag =0; // Strings are equal
    else
        flag =1; // One string is longer than the other

    return flag;
}
```

5. Suppose a File “city.txt” contains the record of today’s temperature of various cities. It would look something like:

```
Chennai 38
Pilani 42
Goa 44
Mumbai 37
Delhi 41
Hyderabad 40
Jaipur 39
...*
```

Now, the following code finds the temperature of a city whose name is given by a user. The program opens the file and compares each city one by one using a file pointer. Once it finds the city, it then displays its temperature. Fill in the blanks with suitable code. You can use <string.h> library. **4M**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char name[50], m[30];
    FILE *fp;
    int k;

    fp = fopen("city.txt", "r");
    printf("Enter the name of a city: ");
    scanf("%s", name);

    while( fscanf(fp, "%s %d", m, &k) != EOF )
    {
        _____ Empty _____;
        if (strcmp(name, m)==0)
        {
            _____ Empty _____;
            printf(" City=%s \t temp = %d \n", m, k);
        }
    }
    fclose(fp);
    return 0;
}
```

Sample execution:

```
Enter the name of the city: Jaipur
City = Jaipur Temperature = 39
```

6. You are given the following incomplete C code that deletes the middle node of a singly linked list (if the number of nodes is odd), or the second middle node (if even). Complete the code. Note that there is no separate structure to store the header and count. The function only receives the pointer to the first node. **8M**

```
// Node structure.
struct Node {
    int data;
    struct Node* next;
};

void deleteMiddle(struct Node* headRef) {

    struct Node *slow = *headRef, *fast = *headRef;
    struct Node *prev = NULL;

    // Edge case: Empty list or single node
    if (*headRef == NULL || (*headRef)->next == NULL) {
        free(*headRef);
        *headRef = NULL;
        return;
    }

    // Traverse with slow and fast pointers
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        prev = slow;
        slow = slow->next;
    }

    // Now slow points to the middle node
    prev->next = slow->next;
    free(slow);
}
```

Birla Institute of Technology and Science, Pilani
2nd Semester 2024-2025
CS F111 – Computer Programming – Comprehensive Exam – Part C (Open Book)

=====

5 May 2025

Duration: 60 minutes

34 Marks

=====

1. In this question, you will have to perform the sorting and merging of two doubly linked lists (`list1` and `list2`) of employee records to get another merged, sorted doubly linked list (`merged_list`). You can get a glimpse of what each employee record looks like from the program output given below:

```
List 1:
ID: 101, Name: Alice, Join Date: 2020-01-15
ID: 103, Name: Bob, Join Date: 2020-03-10
ID: 105, Name: Charlie, Join Date: 2020-05-20

List 2:
ID: 102, Name: David, Join Date: 2020-02-05
ID: 104, Name: Eve, Join Date: 2020-04-12
ID: 106, Name: Frank, Join Date: 2020-06-01

Merged List:
ID: 101, Name: Alice, Join Date: 2020-01-15
ID: 102, Name: David, Join Date: 2020-02-05
ID: 103, Name: Bob, Join Date: 2020-03-10
ID: 104, Name: Eve, Join Date: 2020-04-12
ID: 105, Name: Charlie, Join Date: 2020-05-20
ID: 106, Name: Frank, Join Date: 2020-06-01
```

You will have to do the following tasks:

- Create necessary structure definitions for storing employee records and building a doubly linked list. **4M**
- Create a function `readDLL()` that accepts data from the user to create a doubly linked list of employee records. The user also specifies the size of the list. This function should then return the DLL created. **6M**
- Create a function `sortDLL()`, which takes a doubly linked list and sorts the list based on the **Join Date**. It should then return the sorted DLL. You can use any sorting technique that you know. **10M**
- Create a function `mergeSortedDLLs()`, which takes two doubly linked lists and merges them into one doubly linked list such that the merged list is also sorted based on the **Join Date**. It should return the merged list. This function should **NOT** use sorting. Merging should be done in-place, meaning you are not allowed to allocate extra memory on the heap to merge two lists. **10M**
- Create a `main()` function that reads two DLLs from the user, sorts them based on the **Join Date**, and merges them into a new DLL sorted based on **Join Date**. It should use the above functions to do so. Then, at the end, it should print all three lists as shown in the above screenshot. **4M**

You are free to choose the function parameters. You can make all necessary assumptions and state them. Assumptions have to be reasonable.

===== End of Document =====

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure to store employee data
typedef struct Employee {
    int id;
    char name[50];
    char join_date[11]; // Format: YYYY-MM-DD
    struct Employee* prev;
    struct Employee* next;
} Employee;
//Marks: 4/4 - Correct definition for doubly linked list and employee info.

// Function to create a new node
Employee* createNode(int id, char* name, char* date) {
    Employee* newNode = (Employee*)malloc(sizeof(Employee));
    newNode->id = id;
    strcpy(newNode->name, name);
    strcpy(newNode->join_date, date);
    newNode->prev = newNode->next = NULL;
    return newNode;
}

// Function to read DLL from user
Employee* readDLL(int size) {
    Employee *head = NULL, *tail = NULL;
    for (int i = 0; i < size; i++) {
        int id;
        char name[50], date[11];
        printf("Enter ID, Name, Join Date (YYYY-MM-DD): ");
        scanf("%d, %49[^\n], %10s", &id, name, date);
        Employee* node = createNode(id, name, date);
        if (!head) {
            head = tail = node;
        } else {
            tail->next = node;
            node->prev = tail;
            tail = node;
        }
    }
    return head;
}
//Marks: 6/6 - Creates and links DLL nodes from user input.

// Function to compare join dates
int compareDate(char* d1, char* d2) {
    return strcmp(d1, d2);
}

```

```

// Function to sort a DLL using bubble sort
Employee* sortDLL(Employee* head) {
    if (!head) return head;
    int swapped;
    Employee *ptr1, *lptr = NULL;
    do {
        swapped = 0;
        ptr1 = head;
        while (ptr1->next != lptr) {
            if (compareDate(ptr1->join_date, ptr1->next->join_date) > 0) {
                // Swap data
                int temp_id = ptr1->id;
                char temp_name[50], temp_date[11];
                strcpy(temp_name, ptr1->name);
                strcpy(temp_date, ptr1->join_date);

                ptr1->id = ptr1->next->id;
                strcpy(ptr1->name, ptr1->next->name);
                strcpy(ptr1->join_date, ptr1->next->join_date);

                ptr1->next->id = temp_id;
                strcpy(ptr1->next->name, temp_name);
                strcpy(ptr1->next->join_date, temp_date);

                swapped = 1;
            }
            ptr1 = ptr1->next;
        }
        lptr = ptr1;
    } while (swapped);
    return head;
}

// Marks: 10/10 - Bubble sort (or any sorting technique) with date comparison and
in-place swapping.

// Merge two sorted DLLs in-place
Employee* mergeSortedDLLs(Employee* first, Employee* second) {
    if (!first) return second;
    if (!second) return first;

    Employee* merged = NULL;

    if (compareDate(first->join_date, second->join_date) <= 0) {
        merged = first;
        merged->next = mergeSortedDLLs(first->next, second);
        if (merged->next) merged->next->prev = merged;
        merged->prev = NULL;
    } else {
        merged = second;
        merged->next = mergeSortedDLLs(first, second->next);
        if (merged->next) merged->next->prev = merged;
        merged->prev = NULL;
    }
    return merged;
}

// Marks: 10/10 - In-place merge of two sorted DLLs without extra memory.

```

```

// Function to print the DLL
void printList(Employee* head) {
    while (head) {
        printf("ID: %d, Name: %s, Join Date: %s\n", head->id, head->name, head->join_date);
        head = head->next;
    }
}

int main() {
    int size1, size2;
    printf("Enter number of employees in List 1: ");
    scanf("%d", &size1);
    Employee* list1 = readDLL(size1);

    printf("Enter number of employees in List 2: ");
    scanf("%d", &size2);
    Employee* list2 = readDLL(size2);

    list1 = sortDLL(list1);
    list2 = sortDLL(list2);

    Employee* merged = mergeSortedDLLs(list1, list2);

    printf("\nList 1:\n");
    printList(list1);
    printf("\nList 2:\n");
    printList(list2);
    printf("\nMerged List:\n");
    printList(merged);

    return 0;
}
// Marks: 4/4 - Functionally correct, minor caveat with post-merge printing.

```